FILE: .gitignore

```
0001: .venv/
0002: __pycache__/
0003: *.pyc
0004: *.pyo
0005: *.pyd
0006: .Python
0007: .streamlit/
0008: models/reference_index.json
```

FILE: README.md

```
0001: # Indian Cattle & Buffalo Breed Recognition App
0002:
0003: This is a Streamlit app for image-based breed recognition of Indian cattle and buffaloes.
0004: It now supports a fully offline workflow.
0005:
0006: ## How It Works
0007:
0008: - You provide a small labeled reference set of breed images.
0009: - The app extracts visual features locally on your machine.
0010: - If Torch is available locally, it uses an offline CNN feature extractor.
0011: - If Torch is unavailable or fails, it falls back to a built-in handcrafted feature extractor.
0012: - For each breed, it builds a prototype vector (average embedding).
0013: - Uploaded images are matched by cosine similarity against breed prototypes.
0014:
0015: ## Project Structure
0016:
0017: - ‘app.py‘: Streamlit UI
0018: - ‘src/breed_recognition.py‘: Recognition engine
0019: - ‘scripts/build_index.py‘: Build reference index from images
0020: - ‘data/reference_images/‘: Put breed-wise folders and images here
0021: - ‘models/reference_index.json‘: Generated index file
0022:
0023: ## Quick Start
0024:
0025: 1. Create and activate virtual environment (recommended).
0026: 2. Install dependencies:
0027:
0028: ‘‘‘bash
0029: pip install -r requirements.txt
0030: ‘‘‘
0031:
0032: 3. Add reference images:
0033:
0034: ‘‘‘text
```

````
0035: data/reference_images/
0036:   Gir/
0037:     img1.jpg
0038:     img2.jpg
0039:   Sahiwal/
0040:     img1.jpg
0041:   Murrah/
0042:     img1.jpg
0043: ```
0044:
0045: 4. Build the index:
0046:
0047: ```bash
0048: python -m scripts.build_index
0049: ```
0050:
0051: 5. Run the app:
0052:
0053: ```bash
0054: streamlit run app.py
0055: ```
0056:
0057: ## Offline Use
0058:
0059: - No internet is required to run predictions once dependencies are installed.
0060: - Add breed images into `data/reference_images/<breed_name>/`.
0061: - The app can auto-build the index on startup when deployed.
0062: - For local development, you can still rebuild manually after adding or changing dataset images
      :
0063:
0064: ```bash
0065: python -m scripts.build_index
0066: ```
0067:
0068: ## Notes
0069:
0070: - Prediction quality depends heavily on reference image quality and diversity.
0071: - The handcrafted offline mode is lightweight and reliable, but usually less accurate than a tr
      ained classifier.
0072: - This is a baseline similarity system, not a fully supervised production classifier.
````

FILE: requirements.txt

```
0001: streamlit==1.41.1
0002: torch>=2.2.0
0003: torchvision>=0.17.0
0004: Pillow>=10.3.0
```

```
0005: numpy>=1.26.0


FILE: app.py

0001: from __future__ import annotations
0002:
0003: from pathlib import Path
0004:
0005: import streamlit as st
0006: from PIL import Image
0007:
0008: from scripts.build_index import build_reference_index, collect_images
0009: from src.breed_recognition import BreedRecognizer
0010:
0011:
0012: LANGUAGE_OPTIONS = {
0013:     "English": "en",
0014:     "??????????????": "hi",
0015:     "??????????????": "mr",
0016: }
0017:
0018: TEXT = {
0019:     "en": {
0020:         "page_title": "PASHUPAHECHAN",
0021:         "lang": "Language",
0022:         "hero_kicker": "Indian Livestock Vision",
0023:         "hero_title": "????????????????????????",
0024:         "hero_copy": "Breed recognition for Indian cattle and buffaloes with fast offline predi
      ctions for field teams, students, and livestock extension workers.",
0025:         "indexed_breeds": "Indexed Breeds",
0026:         "indexed_breeds_sub": "Breed folders currently ready for matching.",
0027:         "recognition_mode": "Recognition Mode",
0028:         "recognition_mode_sub": "Runs offline and remains usable even with poor connectivity.",
0029:         "index_status": "Index Status",
0030:         "index_status_sub": "Reference images are checked and indexed automatically.",
0031:         "rebuilt": "Rebuilt at startup",
0032:         "ready": "Ready for prediction",
0033:         "waiting": "Waiting for dataset",
0034:         "supported": "Supported breeds",
0035:         "supported_copy": "Starter support includes major Indian cattle and buffalo breeds.",
0036:         "workflow_note": "Add more images in data/reference_images/<breed_name>/. The app rebui
      lds its breed index automatically on startup.",
0037:         "upload_title": "Upload and analyze",
0038:         "upload_copy": "Use a clear side-view or near-profile animal image. Similar lighting an
      d less background clutter improve matching.",
0039:         "upload_label": "Upload image",
0040:         "prediction_count": "Number of predictions",
```

```
0041:         "upload_caption": "Uploaded field image",
0042:         "predict": "Predict Breed",
0043:         "spinner": "Analyzing image and comparing with reference breeds...",
0044:         "no_index": "No usable reference index was found. Add breed images to data/reference_im
      ages/<breed_name>/ and restart the app.",
0045:         "most_likely": "Most likely match",
0046:         "confidence": "Confidence",
0047:         "match_confidence": "match confidence",
0048:         "insight_title": "Signature field clue",
0049:         "upload_prompt": "Upload a cattle or buffalo image to begin recognition.",
0050:         "field_guidance": "Field guidance",
0051:         "field_guidance_copy": "This baseline app compares images against your local breed libr
      ary. It works best when each breed folder has multiple clear examples from similar angles. For
      higher accuracy, the next step would be training a dedicated classifier on a larger labeled dat
      aset.",
0052:         "offline_cnn": "Offline mode active: local CNN feature extractor loaded.",
0053:         "offline_hand": "Offline mode active: built-in handcrafted image features loaded. No in
      ternet required.",
0054:         "backend_cnn": "Local CNN Extractor",
0055:         "backend_hand": "Handcrafted Offline Extractor",
0056:         "breed_fallback": "More breed information can be added here.",
0057:     },
0058:     "hi": {
0059:         "page_title": "??????????????????????",
0060:         "lang": "???????????",
0061:         "hero_kicker": "????????????????? ????????????? ?????????????????",
0062:         "hero_title": "??????????????????????",
0063:         "hero_copy": "????????????????? ????????????? ?????? ????????????????? ?????? ?????
      ??????? ????????????? ?????? ????????? ????????? ????????????????? ?????????????????, ?????
      ? ????????????? ?????????, ?????????????????????? ?????? ?????????????????????? ?????????????
      ?????????????????????? ?????? ????????? ????????????????? ?????????",
0064:         "indexed_breeds": "???????????????????????? ?????????????????",
0065:         "indexed_breeds_sub": "????????????? ?????? ????????? ????????????????? ???????????
       ????????????????????????",
0066:         "recognition_mode": "????????????? ????????",
0067:         "recognition_mode_sub": "?????? ????????????????? ??????????? ?????? ?????? ????????
      ?????? ??????????????????? ????????? ?????? ????????????????? ??????????? ?????????",
0068:         "index_status": "??????????????????? ?????????????????",
0069:         "index_status_sub": "??????????????????? ??????????? ???????????-?????? ???????????
      ???? ?????? ??????????????????? ?????? ??????????? ????????????",
0070:         "rebuilt": "???????????? ???????????? ????????? ????????? ?????? ????????????? ??????
      ???",
0071:         "ready": "????????????? ?????? ????????? ?????????????",
0072:         "waiting": "??????????????????? ?????? ??????????????????????????",
0073:         "supported": "??????????????????? ?????????????????",
0074:         "supported_copy": "??????????????????? ????????????????? ????????? ???????????????
      ? ???????????????? ????????? ?????? ??????????? ????????????????? ????????????? ?????????
      ???",
```

```
        "workflow_note": "data/reference_images/<breed_name>/ ????????? ?????? ????????????????
????????? ????????????????????? ?????? ???????????? ???????????? ?????? ???????????????????? ??
??????????-?????? ????????? ???????????????",
        "upload_title": "?????????????????? ??????????????? ???????????? ?????? ???????????????
???",
        "upload_copy": "??????????????? ?????? ????????? ???????????? ?????? ?????????????????
?????? ???????????? ???????????? ???????????? ??????????????? ?????? ?????? ??????????????????
???????? ????????? ?????? ?????????????????? ??????????????? ??????????????? ????????????",
        "upload_label": "?????????????????? ??????????????? ????????????",
        "prediction_count": "??????????????? ?????????????????????????????????????? ?????????
???????? ?????????",
        "upload_caption": "??????????????? ?????? ?????? ??????????????????",
        "predict": "???????????? ?????????????????????",
        "spinner": "?????????????????? ?????? ??????????????????????? ???????????? ?????? ????
????? ?????? ?????? ???????????????????? ?????????????????? ?????? ??????????????? ?????? ????
????? ??????...",
        "no_index": "??????????????? ??????????????? ????????????????????? ???????????????????
? ???????????? ??????????????? data/reference_images/<breed_name>/ ????????? ???????????? ?????
? ??????????????????????? ?????????????????? ?????? ?????? ????????? ?????? ???????????? ?????
??????????",
        "most_likely": "???????????? ????????????????????? ???????????????",
        "confidence": "????????????????????? ????????????",
        "match_confidence": "??????????????? ????????????????????? ????????????",
        "insight_title": "??????????????? ????????? ??????????????? ?????? ???????????????",
        "upload_prompt": "??????????????? ???????????? ???????????? ?????? ????????? ?????????
?????? ???????????? ?????? ?????????????????? ??????????????? ???????????????",
        "field_guidance": "?????????????????? ???????????????",
        "field_guidance_copy": "?????? ??????????????????????????? ?????? ???????????? ????????
????????????? ???????????? ???????????????????????????? ?????? ??????????????? ???????????? ????
????? ??????????????? ?????????????????? ?????? ????????? ?????? ???????????? ?????????????????
???? ????????? ?????? ???????????? ??????????????? ?????? ?????? ????????? ????????????????????
???? ??????????????? ???????????? ?????????????????? ?????? ????????? ????????? ???????????? ??
?????????? ?????????????????? ????????????????????? ?????? ????????? ???????????? ?????????????
????????????????? ???????????? ?????? ???????????? ?????????",
        "offline_cnn": "?????????????????? ????????? ?????????????????? ??????: ???????????? CN
N ???????????? ????????????????????????????????????? ????????? ?????? ????????? ?????????",
        "offline_hand": "?????????????????? ????????? ????????????????? ??????: ??????????????
?-?????? ??????????????????????????????????????? ???????????? ???????????? ??????????????? ????
?? ????????? ???????????? ????????????????????? ?????? ??????????????? ???????????? ?????????",
        "backend_cnn": "???????????? CNN ????????????????????????????????????",
        "backend_hand": "?????????????????????????????????????? ?????????????????? ???????????
??????????????????????????",
        "breed_fallback": "???????????? ????????? ????????? ?????? ???????????? ???????????????
?????? ??????????????? ?????? ???????????? ?????????",
    },
    "mr": {
        "page_title": "????????????????????????",
        "lang": "????????????",
```

```
0100:         "hero_kicker": "???????????????? ?????????????? ????????????????",
0101:         "hero_title": "??????????????????????????",
0102:         "hero_copy": "???????????????????? ???????????? ????????? ?????????????????????????????????
      ??????????? ?????????????????????????????????? ????????? ???????????????????? ?????????????????, ??
      ???? ????????????????????, ?????????????????????????????????? ????????? ?????????????????????? ???????
      ?????????????????????????????????????????????? ?????????????????????? ?????????.",
0103:         "indexed_breeds": "?????????????????????????????? ???????????",
0104:         "indexed_breeds_sub": "?????????????????????????????? ???????????????????? ????????????????????
      ???? ????????????????????.",
0105:         "recognition_mode": "????????? ?????????",
0106:         "recognition_mode_sub": "?????? ????????? ???????????????????? ?????????????????? ?????????
      ????????? ?????????????????????????????????????????????? ???????????????????? ??????????????????.",
0107:         "index_status": "?????????????????????? ????????????????????",
0108:         "index_status_sub": "?????????????????????? ??????????? ?????????????????? ??????????????????
      ??? ?????????????????? ????????? ?????????????????????? ??????????? ???????????.",
0109:         "rebuilt": "?????????????????????????????? ???????????????????? ???????????",
0110:         "ready": "?????????????????????????????? ????????????",
0111:         "waiting": "?????????????????????????????? ????????? ??????????? ?????????",
0112:         "supported": "?????????????????????? ????????????",
0113:         "supported_copy": "??????????????????????????? ??????????????????????????? ????????????????????
       ????????????????????? ??????????? ????????? ?????????????????????????????????? ??????????? ????????
      ????.",
0114:         "workflow_note": "data/reference_images/<breed_name>/ ?????????????????? ??????????? ????
      ???????? ????????????. ????????? ??????????? ?????????????????? ?????????????????????? ??????????
      ???? ??????????? ????????????.",
0115:         "upload_title": "???????????? ?????????????????? ????????? ????????? ??????????????????",
0116:         "upload_copy": "?????????????????????????????????? ???????????????????? ???????????????????? ??
      ????????????? ?????????????????????????? ???????????? ??????????????????. ?????????????????? ??????????
      ?????????????????????????????????????? ????????? ????????? ????????????????????????????????????? ????????
      ??????? ???????????????????? ???????????????????? ?????????????????????????????.",
0117:         "upload_label": "???????????? ?????????????????? ?????????",
0118:         "prediction_count": "???????????? ?????????????????? ?????????????????????????",
0119:         "upload_caption": "?????????????????? ???????????????????? ????????????",
0120:         "predict": "????????? ????????????",
0121:         "spinner": "???????????????????? ?????????????????????????? ???????????? ????????????????????
      ??? ??????????????????????? ?????????????????? ???????????? ????????? ?????????...",
0122:         "no_index": "???????????????????? ??????????????????????? ??????????????????????? ??????????????
      ??????? ????????????. data/reference_images/<breed_name>/ ?????????????????? ???????????????????? ??
      ?????????? ???????????? ????????? ????????? ???????????????????? ??????????? ?????????.",
0123:         "most_likely": "???????????????????? ??????????????????????? ??????????????????",
0124:         "confidence": "??????????????????????? ??????????????????",
0125:         "match_confidence": "?????????????????? ??????????????????????? ???????????????????",
0126:         "insight_title": "???????????????????? ???????????????????? ??????????????????",
0127:         "upload_prompt": "????????? ???????????? ?????????????????????????????????? ????????? ?????
      ?????????? ??????????????????????? ???????????? ?????????????????? ?????????.",
0128:         "field_guidance": "???????????????????? ?????????????????????????????????",
0129:         "field_guidance_copy": "?????? ?????????????????????????? ????????? ??????????????????????
       ????????????????????? ?????????????????????????????? ?????????????????????????????????? ?????????????
```

?? ???????????. ?????????????????????? ????????????????????????????????????? ?????????????????
???????? ?????????????????????? ???????????????????????????????? ??????????????????????????
 ????????????????????? ??????????? ???????????????????? ??????????? ???????????. ???????????
?????????????????????????????? ????????????????? ?????????????????????????? ???????????????????? ????
????????????????? ?????????????????????????????? ?????????????????????????? ????????????????? ??????????
????????????????????? ??????????? ???????????.",
0130:   "offline_cnn": "????????????????????? ????????? ?????????????????????: ??????????? CNN ?????
??????? ?? ????????? ??????????? ?????????.",
0131:   "offline_hand": "????????????????????? ????????? ?????????????????????: ?????????????????? ??
?????????????????????????????????????? ??????????? ???????????????????? ???????????????????? ?????????
 ???????????. ?????????????????????????????? ????????? ???????????.",
0132:   "backend_cnn": "??????????? CNN ??",
0133:   "backend_hand": "??? ???????????????????? ?????????????????
????????????????????????????",
0134:   "breed_fallback": "??????????? ??????????? ??????????? ???????????????????? ?????????????
???????? ?????????????????? ???????????.",
0135:   },
0136: }
0137:
0138: INDIAN_CATTLE_BREEDS = [
0139:   "Gir",
0140:   "Sahiwal",
0141:   "Rathi",
0142:   "Tharparkar",
0143:   "Red Sindhi",
0144:   "Kankrej",
0145:   "Ongole",
0146:   "Hariana",
0147:   "Hallikar",
0148:   "Deoni",
0149:]
0150:
0151: INDIAN_BUFFALO_BREEDS = [
0152:   "Murrah",
0153:   "Jaffarabadi",
0154:   "Mehsana",
0155:   "Bhadawari",
0156:   "Surti",
0157:   "Nili-Ravi",
0158:   "Pandharpuri",
0159:   "Nagpuri",
0160:   "Toda",
0161:]
0162:
0163: BREED_DETAILS = {
0164:   "Gir": "Heat-tolerant dairy breed from Gujarat, known for a domed forehead and long ears.",
0165:   "Sahiwal": "Strong milch breed with a reddish-brown coat, common in North India and Punjab
regions.",

```
    "Rathi": "Dual-purpose cattle from Rajasthan, valued for both milk yield and hardiness.",
    "Tharparkar": "Desert-adapted breed recognized for endurance and pale grey-white coloring."
,
    "Red Sindhi": "Deep reddish dairy breed suited to hot climates and low-input systems.",
    "Kankrej": "Large and powerful breed from western India, often used for both milk and draug
ht work.",
    "Ongole": "Tall, muscular cattle breed from Andhra Pradesh with prominent hump structure.",
    "Hariana": "North Indian breed appreciated for draught strength and moderate milk productio
n.",
    "Hallikar": "Classic southern draught breed, usually lean, agile, and energetic.",
    "Deoni": "Spotted dual-purpose cattle from the Deccan region, productive under village cond
itions.",
    "Murrah": "Premier Indian buffalo breed, jet black, tightly curled horns, and high milk pot
ential.",
    "Jaffarabadi": "Heavy buffalo breed with massive body frame and drooping horns.",
    "Mehsana": "Buffalo breed from Gujarat, often selected for stable milk yield.",
    "Bhadawari": "Copper-toned buffalo known for rich-fat milk and adaptation to hot plains.",
    "Surti": "Medium-sized buffalo with sickle-shaped horns and a calm dairy temperament.",
    "Nili-Ravi": "Large river buffalo breed with a strong dairy profile and characteristic whit
e markings.",
    "Pandharpuri": "Maharashtra buffalo breed with long sword-like horns and elongated face.",
    "Nagpuri": "Hardy buffalo breed used in dry regions, recognized for long flat horns.",
    "Toda": "Nilgiri hill buffalo associated with the Toda community and adapted to upland cond
itions.",
}

BREED_INSIGHTS = {
    "Gir": {
        "trait": "Long pendulous ears with a prominent domed forehead are strong visual markers
.",
        "value": "Well known for dairy strength and heat tolerance in hot regions.",
    },
    "Sahiwal": {
        "trait": "Reddish coat tone with a deep body and calm dairy-type build.",
        "value": "One of the most respected indigenous dairy breeds in the subcontinent.",
    },
    "Murrah": {
        "trait": "Jet-black body with tightly curled horns is the classic Murrah marker.",
        "value": "Top Indian buffalo breed for high milk production potential.",
    },
    "Kankrej": {
        "trait": "Large, powerful frame with a characteristic lyre-shaped horn profile.",
        "value": "Strong dual-purpose breed with both milk and draught importance.",
    },
    "Pandharpuri": {
        "trait": "Very long sword-like horns make this buffalo visually striking.",
        "value": "Adapted to regional conditions in Maharashtra.",
    },
```

```
}

REFERENCE_ROOT = Path("data/reference_images")
INDEX_PATH = Path("models/reference_index.json")


def needs_index_rebuild(reference_root: Path, index_path: Path) -> bool:
    if not index_path.exists():
        return True
    image_files = collect_images(reference_root) if reference_root.exists() else []
    if not image_files:
        return False
    latest_image_time = max(path.stat().st_mtime for path in image_files)
    return index_path.stat().st_mtime < latest_image_time


def ensure_reference_index() -> tuple[bool, int]:
    if not REFERENCE_ROOT.exists():
        return False, 0
    image_files = collect_images(REFERENCE_ROOT)
    if not image_files:
        return False, 0
    if needs_index_rebuild(REFERENCE_ROOT, INDEX_PATH):
        index = build_reference_index(REFERENCE_ROOT, INDEX_PATH)
        return True, len(index)
    recognizer = BreedRecognizer(index_path=str(INDEX_PATH))
    return False, len(recognizer.reference_vectors)


def render_hero_art() -> str:
    return """
    <svg viewBox="0 0 640 420" xmlns="http://www.w3.org/2000/svg" role="img" aria-label="Cow st
anding in an Indian farm field">
      <defs>
        <linearGradient id="sky" x1="0" x2="0" y1="0" y2="1">
          <stop offset="0%" stop-color="#82cfff"/>
          <stop offset="100%" stop-color="#fff6dd"/>
        </linearGradient>
        <linearGradient id="hill" x1="0" x2="1" y1="0" y2="1">
          <stop offset="0%" stop-color="#90c56d"/>
          <stop offset="100%" stop-color="#4f9650"/>
        </linearGradient>
        <linearGradient id="field" x1="0" x2="1" y1="0" y2="1">
          <stop offset="0%" stop-color="#d2b25f"/>
          <stop offset="100%" stop-color="#7ca348"/>
        </linearGradient>
        <linearGradient id="cowBody" x1="0" x2="1" y1="0" y2="1">
          <stop offset="0%" stop-color="#fff7ef"/>
```

```
          <stop offset="100%" stop-color="#f1ddc8"/>
        </linearGradient>
      </defs>
      <rect width="640" height="420" rx="34" fill="url(#sky)"/>
      <circle cx="528" cy="84" r="42" fill="#ffd25e"/>
      <path d="M0 236C104 194 185 220 254 206C340 189 401 145 484 156C552 166 600 206 640 194V420H0Z" fill="url(#hill)"/>
      <path d="M0 295C88 279 144 306 230 295C305 286 360 250 428 248C511 246 568 284 640 270V420H0Z" fill="url(#field)"/>
      <path d="M0 330C81 311 145 340 210 322C278 303 334 280 420 293C508 307 566 344 640 325V420H0Z" fill="#6d9d43" opacity="0.75"/>
      <g transform="translate(150 165)">
        <ellipse cx="158" cy="120" rx="132" ry="78" fill="url(#cowBody)"/>
        <ellipse cx="259" cy="86" rx="56" ry="49" fill="url(#cowBody)"/>
        <path d="M248 51L276 28L271 62Z" fill="#be8750"/>
        <path d="M224 54L193 30L207 68Z" fill="#be8750"/>
        <circle cx="236" cy="82" r="5.5" fill="#182447"/>
        <path d="M116 82C130 59 171 59 184 83C165 99 136 104 116 82Z" fill="#c98546"/>
        <path d="M172 128C187 111 223 112 233 142C212 157 184 155 172 128Z" fill="#c98546"/>
        <path d="M87 137C97 118 125 116 140 135C123 149 100 151 87 137Z" fill="#c98546"/>
        <rect x="96" y="191" width="18" height="108" rx="9" fill="#7b5b44"/>
        <rect x="158" y="193" width="18" height="106" rx="9" fill="#7b5b44"/>
        <rect x="236" y="186" width="18" height="113" rx="9" fill="#7b5b44"/>
        <rect x="286" y="182" width="18" height="117" rx="9" fill="#7b5b44"/>
      </g>
    </svg>
    """


def inject_styles() -> None:
    st.markdown(
        """
        <style>
        .stApp {
            background:
                radial-gradient(circle at top right, rgba(244, 185, 66, 0.30), transparent 26%),
                radial-gradient(circle at left 18%, rgba(75, 163, 242, 0.18), transparent 24%),
                radial-gradient(circle at bottom right, rgba(217, 79, 112, 0.16), transparent 22%),
                linear-gradient(180deg, #fff7e7 0%, #fffdf7 46%, #f8f0ff 100%);
        }
        .block-container {
            padding-top: 1.5rem;
            padding-bottom: 3rem;
        }
        .hero-shell, .panel-shell, .metric-card, .hero-art-shell, .result-hero, .insight-card {
            border-radius: 24px;
```

```
            border: 1px solid rgba(27, 36, 64, 0.09);
            box-shadow: 0 12px 32px rgba(29, 41, 64, 0.08);
        }
        .hero-shell, .panel-shell, .metric-card {
            background: linear-gradient(180deg, rgba(255,255,255,0.92), rgba(255,248,236,0.86))
    ;
            padding: 1.3rem;
        }
        .hero-art-shell {
            background: linear-gradient(180deg, rgba(255,255,255,0.96), rgba(255,248,236,0.90))
    ;
            padding: 0.7rem;
        }
        .hero-kicker {
            display: inline-block;
            padding: 0.4rem 0.8rem;
            border-radius: 999px;
            background: linear-gradient(135deg, rgba(47,125,79,0.14), rgba(75,163,242,0.12));
            font-size: 0.8rem;
            font-weight: 800;
            letter-spacing: 0.14em;
            text-transform: uppercase;
            color: #2f7d4f;
        }
        .hero-title {
            font-size: clamp(2.2rem, 4vw, 4rem);
            font-weight: 800;
            color: #17213d;
            margin: 0.8rem 0;
        }
        .hero-copy, .section-copy, .metric-sub, .result-copy, .insight-value {
            color: #5d6486;
            line-height: 1.6;
        }
        .metric-label, .result-label, .insight-title {
            color: #5d6486;
            font-size: 0.82rem;
            text-transform: uppercase;
            letter-spacing: 0.12em;
            font-weight: 700;
        }
        .metric-value, .result-breed {
            color: #182447;
            font-weight: 800;
        }
        .metric-value {
            font-size: 1.7rem;
            margin-top: 0.35rem;
```

```
0342:         }
0343:         .section-title {
0344:             font-size: 1.1rem;
0345:             font-weight: 800;
0346:             color: #1b2440;
0347:         }
0348:         .breed-chip {
0349:             display: inline-block;
0350:             margin: 0.25rem 0.35rem 0 0;
0351:             padding: 0.45rem 0.7rem;
0352:             border-radius: 999px;
0353:             background: linear-gradient(135deg, rgba(75,163,242,0.14), rgba(217,79,112,0.14));
0354:             color: #1b2440;
0355:             font-size: 0.88rem;
0356:             font-weight: 600;
0357:         }
0358:         .workflow-note {
0359:             margin-top: 0.8rem;
0360:             padding: 0.85rem 1rem;
0361:             border-left: 4px solid #ef6a3c;
0362:             background: linear-gradient(135deg, rgba(239,106,60,0.10), rgba(244,185,66,0.12));
0363:             border-radius: 12px;
0364:         }
0365:         .result-hero {
0366:             background: linear-gradient(135deg, rgba(47,125,79,0.15), rgba(244,185,66,0.24), rg
      ba(75,163,242,0.14));
0367:             padding: 1.2rem 1.25rem;
0368:             margin-bottom: 1rem;
0369:         }
0370:         .insight-card {
0371:             background: linear-gradient(135deg, rgba(75,163,242,0.14), rgba(244,185,66,0.22), r
      gba(47,125,79,0.14));
0372:             padding: 1rem 1.1rem;
0373:             margin: 0.85rem 0 1rem 0;
0374:         }
0375:         .stButton > button {
0376:             background: linear-gradient(135deg, #ef6a3c, #d94f70, #f4b942);
0377:             color: white;
0378:             border: none;
0379:             border-radius: 14px;
0380:             font-weight: 800;
0381:         }
0382:         div[data-testid="stProgressBar"] > div > div > div {
0383:             background: linear-gradient(90deg, #2f7d4f, #4ba3f2, #f4b942);
0384:         }
0385:         </style>
0386:         """,
0387:         unsafe_allow_html=True,
```

```
    )


st.set_page_config(page_title="??????????????????????", layout="wide")
inject_styles()

language_name = st.selectbox("Language", options=list(LANGUAGE_OPTIONS.keys()), index=0)
t = TEXT[LANGUAGE_OPTIONS[language_name]]

rebuilt_index, indexed_breeds = ensure_reference_index()

if "recognizer" not in st.session_state:
    st.session_state.recognizer = BreedRecognizer(index_path=str(INDEX_PATH))
elif rebuilt_index:
    st.session_state.recognizer = BreedRecognizer(index_path=str(INDEX_PATH))

recognizer: BreedRecognizer = st.session_state.recognizer
backend_label = t["backend_cnn"] if recognizer.backend == "offline-cnn" else t["backend_hand"]
index_status = t["rebuilt"] if rebuilt_index else t["ready"] if indexed_breeds > 0 else t["wait
ing"]

hero_col, art_col = st.columns([1.2, 0.95], gap="large")
with hero_col:
    st.markdown(
        f"""
        <div class="hero-shell">
            <div class="hero-kicker">{t["hero_kicker"]}</div>
            <div class="hero-title">{t["hero_title"]}</div>
            <div class="hero-copy">{t["hero_copy"]}</div>
        </div>
        """,
        unsafe_allow_html=True,
    )
with art_col:
    st.markdown(f'<div class="hero-art-shell">{render_hero_art()}</div>', unsafe_allow_html=Tru
e)

metric_col1, metric_col2, metric_col3 = st.columns(3)
with metric_col1:
    st.markdown(
        f"""
        <div class="metric-card">
            <div class="metric-label">{t["indexed_breeds"]}</div>
            <div class="metric-value">{indexed_breeds}</div>
            <div class="metric-sub">{t["indexed_breeds_sub"]}</div>
        </div>
        """,
        unsafe_allow_html=True,
```

```
    )
with metric_col2:
    st.markdown(
        f"""
        <div class="metric-card">
            <div class="metric-label">{t["recognition_mode"]}</div>
            <div class="metric-value" style="font-size:1.25rem;">{backend_label}</div>
            <div class="metric-sub">{t["recognition_mode_sub"]}</div>
        </div>
        """,
        unsafe_allow_html=True,
    )
with metric_col3:
    st.markdown(
        f"""
        <div class="metric-card">
            <div class="metric-label">{t["index_status"]}</div>
            <div class="metric-value" style="font-size:1.25rem;">{index_status}</div>
            <div class="metric-sub">{t["index_status_sub"]}</div>
        </div>
        """,
        unsafe_allow_html=True,
    )

if recognizer.backend == "offline-cnn":
    st.success(t["offline_cnn"])
else:
    st.warning(t["offline_hand"])

main_col, side_col = st.columns([1.45, 0.9], gap="large")

with side_col:
    st.markdown('<div class="panel-shell">', unsafe_allow_html=True)
    st.markdown(f'<div class="section-title">{t["supported"]}</div>', unsafe_allow_html=True)
    st.markdown(f'<div class="section-copy">{t["supported_copy"]}</div>', unsafe_allow_html=Tru
e)
    st.markdown("".join(f'<span class="breed-chip">{breed}</span>' for breed in INDIAN_CATTLE_B
REEDS), unsafe_allow_html=True)
    st.markdown("".join(f'<span class="breed-chip">{breed}</span>' for breed in INDIAN_BUFFALO_
BREEDS), unsafe_allow_html=True)
    st.markdown(f'<div class="workflow-note">{t["workflow_note"]}</div>', unsafe_allow_html=Tru
e)
    st.markdown("</div>", unsafe_allow_html=True)

with main_col:
    st.markdown('<div class="panel-shell">', unsafe_allow_html=True)
    st.markdown(f'<div class="section-title">{t["upload_title"]}</div>', unsafe_allow_html=True
)
```

```
0477:     st.markdown(f'<div class="section-copy">{t["upload_copy"]}</div>', unsafe_allow_html=True)
0478:
0479:     uploaded = st.file_uploader(t["upload_label"], type=["jpg", "jpeg", "png", "webp"])
0480:     top_k = st.slider(t["prediction_count"], min_value=1, max_value=5, value=3)
0481:
0482:     if uploaded is not None:
0483:         image = Image.open(uploaded)
0484:         st.image(image, caption=t["upload_caption"], use_container_width=True)
0485:
0486:         if st.button(t["predict"], type="primary"):
0487:             with st.spinner(t["spinner"]):
0488:                 predictions = recognizer.predict(image, top_k=top_k)
0489:
0490:             if not predictions:
0491:                 st.error(t["no_index"])
0492:             else:
0493:                 best = predictions[0]
0494:                 insight = BREED_INSIGHTS.get(best.breed)
0495:                 st.markdown(
0496:                     f"""
0497:                     <div class="result-hero">
0498:                         <div class="result-label">{t["most_likely"]}</div>
0499:                         <div class="result-breed">{best.breed}</div>
0500:                         <div class="result-copy">
0501:                             {t["confidence"]}: {best.confidence * 100:.1f}%<br>
0502:                             {BREED_DETAILS.get(best.breed, t["breed_fallback"])}
0503:                         </div>
0504:                     </div>
0505:                     """,
0506:                     unsafe_allow_html=True,
0507:                 )
0508:
0509:                 if insight:
0510:                     st.markdown(
0511:                         f"""
0512:                         <div class="insight-card">
0513:                             <div class="insight-title">{t["insight_title"]}</div>
0514:                             <div class="insight-trait">{insight["trait"]}</div>
0515:                             <div class="insight-value">{insight["value"]}</div>
0516:                         </div>
0517:                         """,
0518:                         unsafe_allow_html=True,
0519:                     )
0520:
0521:                 for idx, pred in enumerate(predictions, start=1):
0522:                     st.markdown(f"**{idx}. {pred.breed}**")
0523:                     st.progress(float(pred.confidence), text=f"{pred.confidence * 100:.1f}% {t[
      'match_confidence']}")
```

```
0524:                     detail = BREED_DETAILS.get(pred.breed)
0525:                     if detail:
0526:                         st.caption(detail)
0527:     else:
0528:         st.info(t["upload_prompt"])
0529:
0530:     st.markdown("</div>", unsafe_allow_html=True)
0531:
0532: st.markdown(
0533:     f"""
0534:     <div class="panel-shell" style="margin-top: 1.3rem;">
0535:         <div class="section-title">{t["field_guidance"]}</div>
0536:         <div class="section-copy">{t["field_guidance_copy"]}</div>
0537:     </div>
0538:     """,
0539:     unsafe_allow_html=True,
0540: )
```

FILE: src\breed_recognition.py

```
0001: from __future__ import annotations
0002:
0003: import json
0004: from dataclasses import dataclass
0005: from pathlib import Path
0006: from typing import Dict, List
0007:
0008: import numpy as np
0009: from PIL import Image, ImageFilter, ImageOps
0010:
0011: try:
0012:     import torch
0013:     import torchvision.models as models
0014:     import torchvision.transforms as transforms
0015: except Exception:
0016:     torch = None
0017:     models = None
0018:     transforms = None
0019:
0020:
0021: @dataclass
0022: class Prediction:
0023:     breed: str
0024:     confidence: float
0025:
0026:
0027: class BreedRecognizer:
```

```
    def __init__(self, index_path: str = "models/reference_index.json") -> None:
        self.index_path = Path(index_path)
        self.reference_vectors: Dict[str, np.ndarray] = {}
        self.backend = "offline-handcrafted"
        self.device = "cpu"
        self.model = None
        self.transform = None
        self._initialize_backend()
        self._load_index()

    def _initialize_backend(self) -> None:
        if torch is None or models is None or transforms is None:
            return

        try:
            self.device = "cuda" if torch.cuda.is_available() else "cpu"
            model = models.efficientnet_b0(weights=None)
            model.classifier = torch.nn.Identity()
            model.eval()
            model.to(self.device)
            self.model = model
            self.transform = transforms.Compose(
                [
                    transforms.Resize((224, 224)),
                    transforms.ToTensor(),
                    transforms.Normalize(
                        mean=[0.485, 0.456, 0.406],
                        std=[0.229, 0.224, 0.225],
                    ),
                ]
            )
            self.backend = "offline-cnn"
        except Exception:
            self.backend = "offline-handcrafted"
            self.device = "cpu"
            self.model = None
            self.transform = None

    def _load_index(self) -> None:
        if not self.index_path.exists():
            self.reference_vectors = {}
            return

        with self.index_path.open("r", encoding="utf-8") as f:
            raw = json.load(f)
        self.reference_vectors = {
            breed: np.array(vector, dtype=np.float32) for breed, vector in raw.items()
        }
```

```
0076:
0077:     def is_ready(self) -> bool:
0078:         return len(self.reference_vectors) > 0
0079:
0080:     def extract_embedding(self, image: Image.Image) -> np.ndarray:
0081:         image = image.convert("RGB")
0082:         if self.backend == "offline-cnn" and self.model is not None and self.transform is not N
      one:
0083:             tensor = self.transform(image).unsqueeze(0).to(self.device)
0084:             with torch.no_grad():
0085:                 features = self.model(tensor).squeeze(0).detach().cpu().numpy()
0086:             return self._normalize(features)
0087:
0088:         return self._extract_handcrafted_features(image)
0089:
0090:     def _extract_handcrafted_features(self, image: Image.Image) -> np.ndarray:
0091:         resized = image.resize((192, 192))
0092:         rgb = np.asarray(resized, dtype=np.float32) / 255.0
0093:
0094:         hist_parts: List[np.ndarray] = []
0095:         for channel in range(3):
0096:             hist, _ = np.histogram(rgb[:, :, channel], bins=16, range=(0.0, 1.0))
0097:             hist_parts.append(hist.astype(np.float32))
0098:
0099:         gray = ImageOps.grayscale(resized)
0100:         edges = gray.filter(ImageFilter.FIND_EDGES)
0101:         edge_array = np.asarray(edges, dtype=np.float32) / 255.0
0102:         edge_hist, _ = np.histogram(edge_array, bins=16, range=(0.0, 1.0))
0103:
0104:         brightness_mean = np.array([edge_array.mean()], dtype=np.float32)
0105:         brightness_std = np.array([edge_array.std()], dtype=np.float32)
0106:
0107:         downsample = gray.resize((16, 16))
0108:         texture = np.asarray(downsample, dtype=np.float32).flatten() / 255.0
0109:
0110:         features = np.concatenate(
0111:             hist_parts
0112:             + [
0113:                 edge_hist.astype(np.float32),
0114:                 brightness_mean,
0115:                 brightness_std,
0116:                 texture.astype(np.float32),
0117:             ]
0118:         )
0119:         return self._normalize(features)
0120:
0121:     @staticmethod
0122:     def _normalize(vector: np.ndarray) -> np.ndarray:
```

```
0123:         norm = np.linalg.norm(vector) + 1e-10
0124:         return vector / norm
0125:
0126:     @staticmethod
0127:     def _cosine_similarity(a: np.ndarray, b: np.ndarray) -> float:
0128:         return float(np.dot(a, b) / ((np.linalg.norm(a) + 1e-10) * (np.linalg.norm(b) + 1e-10))
      )
0129:
0130:     def predict(self, image: Image.Image, top_k: int = 3) -> List[Prediction]:
0131:         if not self.is_ready():
0132:             return []
0133:
0134:         query_embedding = self.extract_embedding(image)
0135:         scores: List[Prediction] = []
0136:         for breed, ref_vector in self.reference_vectors.items():
0137:             if ref_vector.shape != query_embedding.shape:
0138:                 continue
0139:             sim = self._cosine_similarity(query_embedding, ref_vector)
0140:             conf = max(0.0, min(1.0, (sim + 1.0) / 2.0))
0141:             scores.append(Prediction(breed=breed, confidence=conf))
0142:
0143:         scores.sort(key=lambda x: x.confidence, reverse=True)
0144:         return scores[:top_k]
```

FILE: scripts\build_index.py

```
0001: from __future__ import annotations
0002:
0003: import argparse
0004: import json
0005: from pathlib import Path
0006: from typing import Dict, List
0007:
0008: import numpy as np
0009: from PIL import Image
0010:
0011: from src.breed_recognition import BreedRecognizer
0012:
0013:
0014: def collect_images(folder: Path) -> List[Path]:
0015:     exts = {".jpg", ".jpeg", ".png", ".webp"}
0016:     return [p for p in folder.rglob("*") if p.suffix.lower() in exts]
0017:
0018:
0019: def build_reference_index(reference_root: Path, output_file: Path) -> Dict[str, List[float]]:
0020:     recognizer = BreedRecognizer(index_path="models/does_not_need_to_exist.json")
0021:
```

```
    breed_vectors: Dict[str, np.ndarray] = {}
    for breed_dir in sorted([p for p in reference_root.iterdir() if p.is_dir()]):
        images = collect_images(breed_dir)
        if not images:
            continue

        vectors: List[np.ndarray] = []
        for image_path in images:
            try:
                with Image.open(image_path) as img:
                    vectors.append(recognizer.extract_embedding(img))
            except Exception:
                continue

        if not vectors:
            continue

        centroid = np.mean(np.stack(vectors, axis=0), axis=0)
        centroid = centroid / (np.linalg.norm(centroid) + 1e-10)
        breed_vectors[breed_dir.name] = centroid

    serializable = {
        breed: vector.astype(np.float32).tolist() for breed, vector in breed_vectors.items()
    }
    output_file.parent.mkdir(parents=True, exist_ok=True)
    with output_file.open("w", encoding="utf-8") as f:
        json.dump(serializable, f, indent=2)
    return serializable


def main() -> None:
    parser = argparse.ArgumentParser(description="Build reference index for breed recognition."
)
    parser.add_argument(
        "--reference-root",
        type=Path,
        default=Path("data/reference_images"),
        help="Folder containing one subfolder per breed.",
    )
    parser.add_argument(
        "--output",
        type=Path,
        default=Path("models/reference_index.json"),
        help="Where to save the generated reference index.",
    )
    args = parser.parse_args()

    if not args.reference_root.exists():
```

```
0069:         raise FileNotFoundError(f"Reference folder not found: {args.reference_root}")
0070:
0071:     index = build_reference_index(args.reference_root, args.output)
0072:     print(f"Saved {len(index)} breed prototypes to {args.output}")
0073:
0074:
0075: if __name__ == "__main__":
0076:     main()
```

FILE: android_app\pubspec.yaml

```
0001: name: pashupahachan_android
0002: description: Android starter app for Pashupahachan cattle and buffalo breed recognition.
0003: publish_to: "none"
0004: version: 1.0.0+1
0005:
0006: environment:
0007:   sdk: ">=3.3.0 <4.0.0"
0008:
0009: dependencies:
0010:   flutter:
0011:     sdk: flutter
0012:   cupertino_icons: ^1.0.8
0013:   image_picker: ^1.1.2
0014:
0015: dev_dependencies:
0016:   flutter_test:
0017:     sdk: flutter
0018:   flutter_lints: ^5.0.0
0019:
0020: flutter:
0021:   uses-material-design: true
```

FILE: android_app\analysis_options.yaml

```
0001: include: package:flutter_lints/flutter.yaml
```